

# Feedback y Auto-corrección en Agentes LLM — Análisis completo de la Sesión 10 (Módulo 7)

**Fuente base:** "Feedback y Auto-corrección en Agentes LLM — Cómo los agentes aprenden de sus propios errores" ( 10\_feedback\_and\_correction.pdf , 58 diapositivas) — Módulo 7 (Aprendizaje y Mejora), Programa en Diseño e Implementación de Agentes IA, UTEC Posgrado. Dictada por **Dr. Vicente Machaca Arceda** — un docente distinto a Boris Alzamora (Módulos 5-6); el estilo del material cambia en consecuencia: en vez de diapositivas orientadas a producto/demo, este es un **seminario académico densamente citado**, construido explícitamente sobre un *survey* de investigación.

**Nota técnica:** a diferencia de los PDFs de las Sesiones 15/17/18, este tiene **capa de texto completa y limpia** en las 58 páginas — no hizo falta interpretar imágenes; sí se renderizaron ~20 diapositivas clave para verificar que los diagramas (flechas, cajas, relaciones) coinciden con el texto extraído, lo cual se confirmó en todos los casos.

**Hallazgo clave de esta sesión:** casi toda la clase es una **traducción didáctica, diapositiva por diapositiva, de un único paper ancla** — Liu et al., "A Survey on the Feedback Mechanism of LLM-based AI Agents" (IJCAI-25 Survey Track, pp. 10582–10592) — enriquecido con hallazgos posteriores de 2025-2026 (SCoRe, AutoRefine, ERL, y un paper de mayo de 2026 sobre confabulación de memoria) que el propio material usa para **cuestionar críticamente** las promesas del survey principal. Se verificaron externamente las tres citas más importantes (el survey de Liu et al., el *critical survey* de Kamoi et al., y el paper de confabulación de memoria) y las tres existen y dicen exactamente lo que el material afirma.

## 1. Objetivos y estructura de la sesión

**Objetivos declarados** — al final de la sesión, el estudiante podrá:

1. Clasificar mecanismos de *feedback* (retroalimentación) en agentes LLM (*Large Language Model*, modelo de lenguaje de gran escala).
2. Distinguir cuándo la auto-corrección funciona y cuándo no.
3. Evaluar agentes con métricas de resultado y de proceso.
4. Implementar *loops* de auto-corrección con **LangGraph**.

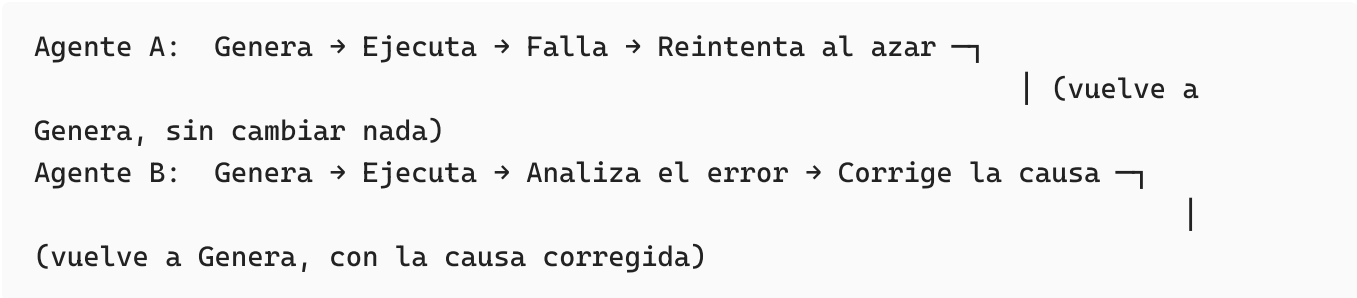
**La observación incómoda que enmarca toda la clase:** "un LLM puede fallar mil veces igual." Sin un mecanismo de *feedback*, repetir una tarea no genera aprendizaje — solo repetición.

**Recorrido de la sesión (4 bloques + práctica + cierre):**

| Bloque                       | Pregunta que responde                                           |
|------------------------------|-----------------------------------------------------------------|
| 1. El problema               | ¿Qué separa a un agente que aprende de uno que solo reintentan? |
| 2. ¿De dónde viene la señal? | Taxonomía de 4 fuentes de <i>feedback</i>                       |
| 3. Los métodos, uno por uno  | 8 métodos concretos, comparados                                 |
| 4. ¿Funciona de verdad?      | Evidencia empírica — y su letra pequeña                         |
| 5. De corregir a mejorarse   | De corregir una salida a reescribir el propio agente            |
| Práctica                     | Dos agentes en LangGraph: Self-Refine vs. Reflexion+intérprete  |
| Cierre                       | Síntesis, direcciones futuras, puente a la Sesión 11            |

## 2. El problema — dos formas de fallar

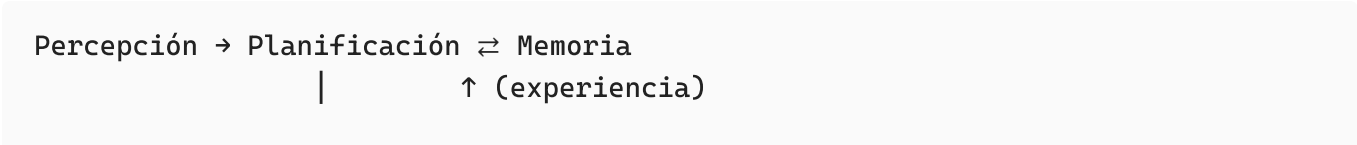
Diapositiva 6 — el diagrama que abre la clase:

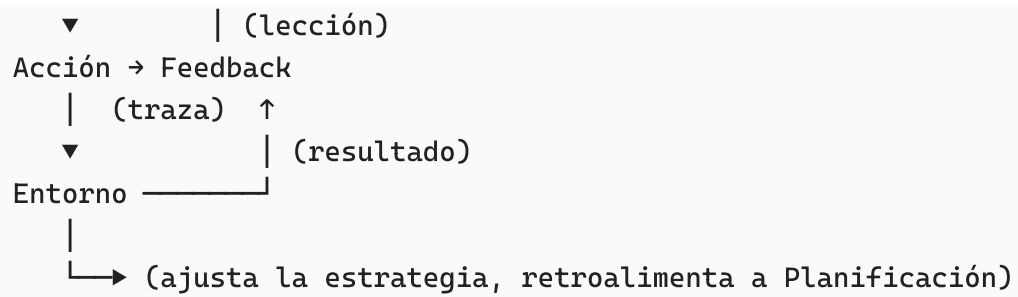


"La diferencia es el *feedback*." El Agente A reintentan sin diagnóstico — puede repetir el mismo error indefinidamente. El Agente B **analiza por qué falló antes de volver a intentar** — la corrección ataca la causa, no solo el síntoma.

### 2.1 El *feedback* como módulo de arquitectura, no como truco de prompt

El material insiste en un punto de diseño importante: el *feedback* no es una frase mágica que se añade a un *prompt* — es un **módulo explícito** dentro de la arquitectura de un agente, acoplado a otros dos módulos:





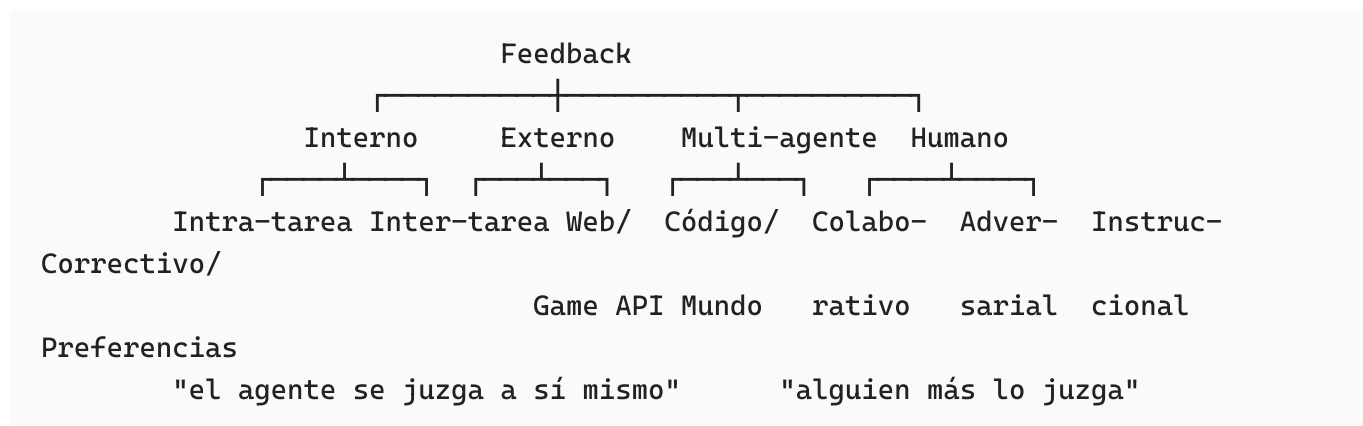
Este es el **marco unificado de 5 módulos** de Liu et al. (IJCAI-25, §2, Fig. 1): Percepción, Planificación, Acción, Memoria y Feedback. El Feedback recibe la **traza** de la Acción y el **resultado** del Entorno, y produce dos salidas: una **lección** que se guarda en Memoria, y un ajuste que retroalimenta directamente a la Planificación.

**Por qué este acoplamiento importa:** al estar conectado a Memoria, una corrección puntual (arreglar un error en esta tarea) puede convertirse en **aprendizaje** (esa lección queda disponible para tareas futuras) — es la diferencia estructural entre los métodos "intra-tarea" y "inter-tarea" que aparecen en la siguiente sección. Pero el módulo, por sí solo, **está vacío sin una señal** — toda la clase gira en torno a de dónde sale esa señal.

## 2.2 Primera discusión en grupo (patrón recurrente de la clase)

El material usa un patrón de discusión de 3 minutos que se repite 4 veces a lo largo de la sesión, siempre sobre la **misma tarea elegida por el grupo**, para que las respuestas se acumulen progresivamente. La primera pregunta ya anticipa el argumento central de la clase: *"si nadie puede detectar el error, ¿tiene sentido automatizar esa tarea?"* — una advertencia temprana de que la auto-corrección presupone que el error **es detectable**, lo cual no es trivial (retomado en el Bloque 4, §5).

## 3. ¿De dónde viene la señal? — taxonomía de 4 fuentes de feedback



El material recorre las 4 fuentes en orden **de la más barata y menos confiable a la más costosa y más confiable** (Liu et al., IJCAI-25, §3.1-3.4).

### 3.1 Fuente 1 — Interno: el agente se juzga solo

|                | Intra-tarea                    | Inter-tarea                   |
|----------------|--------------------------------|-------------------------------|
| Alcance        | La tarea actual                | Múltiples tareas              |
| Velocidad      | Rápida                         | Más lenta                     |
| Memoria        | Corto plazo                    | Largo plazo (persistente)     |
| Generalización | Limitada                       | Alta                          |
| Ejemplos       | ReAct, Self-Refine, AdaPlanner | Reflexion, ExpeL, Retroformer |

En ambos casos el juez es **el propio modelo** — nadie externo verifica si la crítica que el LLM hace de sí mismo es correcta. Ese es "el punto débil común" que el material señala explícitamente antes de pasar a las siguientes fuentes. La memoria persistente del lado "inter-tarea" es, según el propio material, el tema central de la Sesión 11 del programa.

### 3.2 Fuente 2 — Externo: que responda el mundo

Agente → Buscador web (WebGPT) / Game API (Voyager) / Intérprete de código (StepCoder) / Sensores (PaLM-E)

La diferencia cualitativa frente a la Fuente 1: **la señal ya no la inventa el modelo** — "el compilador no alucina." Un test que pasa o falla, una búsqueda que devuelve o no un resultado, un sensor que mide una posición real: son señales **verificables**. El material marca esta idea como la que, al final de la clase, decidirá si un *loop* de auto-corrección sirve o no (retomado en el Bloque 4).

### 3.3 Fuente 3 — Multi-agente: que respondan otros agentes

| Modo         | Mecanismo                                                                   | Ejemplos                    |
|--------------|-----------------------------------------------------------------------------|-----------------------------|
| Colaborativo | Varios agentes con roles complementarios (A1, A2, A3)                       | MetaGPT, InteRecAgent       |
| Adversarial  | Dos agentes debaten frente a un juez, y llegan a consenso por argumentación | Multiagent Debate, ChatEval |

*"Si nadie tiene la verdad, al menos que haya varias perspectivas."* Pero el material advierte de inmediato: más agentes no es estrictamente mejor — hay **tres problemas conocidos** cuando

se agregan agentes:

| Problema     | Descripción                                         | Solución citada                    |
|--------------|-----------------------------------------------------|------------------------------------|
| Coordinación | Discusión sin control                               | Agente coordinador (ChatLLM)       |
| Alineación   | Objetivos en conflicto                              | Descomponer la recompensa (CollaQ) |
| Estabilidad  | Consenso incorrecto (todos se equivocan de acuerdo) | Peer-ranking (PRD)                 |

Estos tres problemas son, según el propio material, "el esqueleto de la Sesión 12" del programa — aquí solo se nombran.

### 3.4 Fuente 4 — Humano: la señal más confiable y la más cara

| Modo          | Ejemplos                                                                                                                           |
|---------------|------------------------------------------------------------------------------------------------------------------------------------|
| Instruccional | WebGPT, InstructGPT                                                                                                                |
| Correctivo    | ReHAC, IBT                                                                                                                         |
| Preferencias  | PrefCLM, RLHF ( <i>Reinforcement Learning from Human Feedback</i> , aprendizaje por refuerzo a partir de retroalimentación humana) |

El problema práctico: *"no escala"* — requiere monitoreo humano continuo. La salida habitual de la industria es sustituir al humano por **LLM-as-a-judge** (usar un LLM para evaluar a otro LLM; Zheng et al., 2023) o **Agent-as-a-judge** (Zhuge et al., 2024) — pero eso, según el material, *"nos devuelve al problema del juez poco confiable"* de la Fuente 1: se cambia un humano caro por un modelo barato que puede estar tan sesgado o equivocado como el agente que evalúa.

#### El costo oculto del feedback humano (diapositiva 17):

| Sesgos                              | Mitigación                             |
|-------------------------------------|----------------------------------------|
| Género, raza, cultura en anotadores | Datasets culturalmente representativos |
| Se amplifican en el modelo          | Anotadores de orígenes diversos        |
| Poca diversidad en los datasets     | Auditorías de equidad                  |

### 3.5 Recap — las cuatro fuentes ordenadas por costo y confiabilidad

| Fuente  | ¿Quién juzga?    | Confiabilidad | Costo |
|---------|------------------|---------------|-------|
| Interno | El propio modelo | Baja          | Bajo  |

| Fuente       | ¿Quién juzga? | Confiabilidad | Costo           |
|--------------|---------------|---------------|-----------------|
| Externo      | El entorno    | <b>Alta</b>   | Medio           |
| Multi-agente | Otros modelos | Media         | Alto            |
| Humano       | Una persona   | <b>Alta</b>   | <b>Muy alto</b> |

**La tensión de todo el diseño, en una frase del material:** *"Lo barato no es confiable; lo confiable no es barato."* Nótese que la fuente **Externa** logra alta confiabilidad a un costo solo medio — es, en la práctica, el mejor punto del compromiso cuando existe (un test, un compilador, una base de datos ya disponibles no cuestan "inventar" un verificador desde cero).

## 4. Los métodos, uno por uno — 8 métodos comparados

El material presenta 8 métodos, ordenados de "sin *feedback*" (línea base) a "con planes/guías" (los más sofisticados), y **luego** muestra que después de 2024 el campo se bifurca en dos direcciones distintas.

### 4.1 Los ocho métodos

| Método                                                        | Qué hace                                                                                                                                                         | Tipo de feedback    | Referencia                  |
|---------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|-----------------------------|
| <b>Act</b>                                                    | Actúa sin razonar (Observación → Acción, se repite)                                                                                                              | Ninguno             | (línea base)                |
| <b>CoT</b> ( <i>Chain-of-Thought</i> , cadena de pensamiento) | Razona en voz alta, pero no toca el entorno                                                                                                                      | Ninguno             | Wei et al., NeurIPS 2022    |
| <b>ReAct</b>                                                  | Alterna Pensamiento → Acción → Observación → Pensamiento                                                                                                         | Interno intra-tarea | Yao et al., ICLR 2023       |
| <b>Self-Refine</b>                                            | El mismo LLM genera, critica y refina su propia salida, iterando sobre la misma respuesta                                                                        | Interno intra-tarea | Madaan et al., NeurIPS 2023 |
| <b>Reflexion</b>                                              | Un Evaluador determina éxito/fallo; el agente escribe <i>por qué</i> falló (auto-reflexión en lenguaje natural) y esa lección se inyecta en el siguiente intento | Interno inter-tarea | Shinn et al., NeurIPS 2023  |
| <b>ExpeL</b> ( <i>Experiential Learner</i> )                  | Compara trayectorias exitosas vs. fallidas y destila reglas ("¿qué hizo                                                                                          | Interno inter-tarea | Zhao et al., AAAI 2024      |

| Método     | Qué hace                                                                                                                                                   | Tipo de feedback           | Referencia               |
|------------|------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------|--------------------------|
|            | distinto el que sí funcionó?")<br>reutilizables en tareas nuevas                                                                                           |                            |                          |
| AdaPlanner | El plan se escribe como código; en cada sub-objetivo verifica si coincide con lo previsto — si no, replanifica                                             | Externo (entorno)          | Sun et al., NeurIPS 2023 |
| AutoGuide  | Extrae de la experiencia <i>offline</i> guías condicionadas al estado ("si el estado es X, entonces haz Y") y recupera solo la que aplica al estado actual | Externo ( <i>offline</i> ) | Fu et al., NeurIPS 2024  |

**Detalle de cada método, con la lógica narrativa del material (cada uno resuelve el punto débil del anterior):**

- **Act vs. ReAct** — la única diferencia es el paso de "Pensamiento". Comparando tasas de éxito: HotpotQA 29%→28% (razonar *no* ayuda en preguntas de un solo salto), WebShop 34%→35% (apenas ayuda), **ALFWorld 28%→40%** (+12 puntos — razonar sí paga en tareas secuenciales largas). Es una evidencia temprana de que el valor del razonamiento depende del tipo de tarea, no es universal.
- **ReAct** — "el pensamiento decide la siguiente acción; la observación corrige el pensamiento." Su límite: *"todo muere al acabar la tarea. No guarda nada."*
- **Self-Refine** — el mismo LLM hace de generador, crítico y refinador. Su punto débil, señalado explícitamente por el material: *"no hay verificador externo: si la crítica se equivoca, el refinamiento empeora la respuesta"* — este detalle anticipa exactamente la evidencia empírica negativa del Bloque 4 (§5).
- **Reflexion** — la idea clave: *"en vez de una recompensa numérica, el agente escribe por qué falló. Ese texto es más informativo que un escalar."* Es el primero de los métodos "que recuerda" — su memoria persiste entre intentos de la misma tarea (y, según cómo se implemente, entre tareas distintas).
- **ExpeL** — a diferencia de Reflexion (que reflexiona sobre *un* fallo), ExpeL extrae patrones comparando **muchas** trayectorias previas, exitosas y fallidas, y los reutiliza.
- **AdaPlanner** — "el prompt en estilo código fuerza a descomponer en sub-objetivos y reduce las alucinaciones del planificador." Es el primer método externo de la lista: la señal de "in-plan / out-of-plan" viene de comparar la ejecución real contra el plan, no de que el LLM se autoevalúe.
- **AutoGuide** — "comprime la experiencia en reglas breves en lenguaje natural, y solo inyecta en el prompt la que aplica ahora" — resuelve el problema de que la memoria acumulada no quepa entera en el contexto.

## 4.2 Después de 2024, el campo se parte en dos (diapositiva 29)

|                                         |                                            |
|-----------------------------------------|--------------------------------------------|
| 2023: ReAct, Self-Refine<br>AdaPlanner) | 2024: ExpeL, AutoGuide (además: Reflexion, |
|                                         | 2025: SCoRe, AutoRefine                    |
|                                         | 2026: ERL                                  |

Esta es, según nuestra lectura, la diapositiva más importante de todo el material para entender **por dónde sigue evolucionando el campo**: los métodos posteriores a 2024 dejan de ser un continuo y se dividen en dos categorías fundamentalmente distintas.

### Andamiaje ( $\Sigma$ ) — son agentes:

- **ERL** (*Experiential Reflective Learning*, 2026): reflexiona, guarda heurísticas **con condición de disparo** y las puntúa por relevancia antes de inyectarlas. **El modelo no se toca**. Resultado citado: Gaia2 56.1% vs. ReAct 48.3%, ExpeL 50.9%, AutoGuide 50.8%.

### Modelo ( $\theta$ ) — no son agentes:

- **SCoRe** (2025, Google DeepMind) y **AutoRefine** (2025) son métodos de **post-entrenamiento** de LLMs con **RL** (*Reinforcement Learning*, aprendizaje por refuerzo). No añaden andamiaje — enseñan al modelo mismo a corregirse o a refinar lo que recupera.

Esta distinción  $\Sigma$  (andamiaje, *scaffolding*) vs.  $\theta$  (parámetros del modelo) es la que estructura todo el Bloque 5 (§6) — vale la pena retenerla desde aquí.

## 4.3 Los dos métodos que entrenan el modelo (diapositiva 30)

|                  | SCoRe — corregirse                                                                                                                                                                                                                                                                                                                              | AutoRefine — refinar lo recuperado                                                                                                                                                                                                                 |
|------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mecanismo        | RL multi-turno <i>online</i> con datos enteramente auto-generados, sin oráculo ni modelo maestro                                                                                                                                                                                                                                                | Paradigma <i>search-and-refine-during-think</i> : <think> → <search> → <refine> → <answer>                                                                                                                                                         |
| Detalle técnico  | Dos fases: una inicialización que evita el colapso de conducta + un <i>bonus</i> de recompensa que amplifica la corrección. Hallazgo previo que motiva el método: el <b>SFT</b> ( <i>Supervised Fine-Tuning</i> , ajuste fino supervisado) sobre trazas de corrección <b>no basta</b> — produce desajuste de distribución o colapso de conducta | <b>GRPO</b> ( <i>Group Relative Policy Optimization</i> , una variante de RL usada para ajustar el modelo comparando grupos de respuestas) con dos recompensas: acierto de la respuesta final + una recompensa específica sobre el bloque <refine> |
| Resultado citado | (ver Bloque 4, tabla MATH, §5)                                                                                                                                                                                                                                                                                                                  | Qwen2.5-3B, Avg QA: 0.405 vs. Search-R1 0.336 (+6,9 puntos), sobre                                                                                                                                                                                 |



|  |                           |                                                                                           |
|--|---------------------------|-------------------------------------------------------------------------------------------|
|  | <b>SCoRe — corregirse</b> | <b>AutoRefine — refinar lo recuperado</b>                                                 |
|  |                           | todo en preguntas <i>multi-hop</i> (que requieren varios saltos de razonamiento/búsqueda) |

"Los dos operan en la caja azul ( $\theta$ ). Si no puedes reentrenar el modelo, no son opciones para ti" — una advertencia práctica directa: para la inmensa mayoría de equipos que consumen un LLM por API sin acceso a sus pesos, SCoRe y AutoRefine son papers para **leer**, no para **implementar**.

## 5. ¿Funciona de verdad? — evidencia empírica y su letra pequeña

### 5.1 Cómo se mide

|                         | <b>Outcome-based</b>                    | <b>Process-based</b>            |
|-------------------------|-----------------------------------------|---------------------------------|
| <b>Qué mide</b>         | Solo el resultado final (tasa de éxito) | Cada paso intermedio del agente |
| <b>Ejemplos citados</b> | (la mayoría de benchmarks)              | DevAI, AMOR                     |

"Hoy casi todo es outcome-based, y eso esconde dónde falló el agente" — una limitación metodológica que el material señala antes de mostrar cualquier número, para que se lean con esa reserva.

### 5.2 Sobre qué se mide — benchmarks citados

| <b>Dominio</b> | <b>Benchmark</b> | <b>Tamaño</b>               | <b>Evaluación</b> |
|----------------|------------------|-----------------------------|-------------------|
| Razonamiento   | HotpotQA         | 113k pares Q&A              | Outcome           |
| Razonamiento   | FEVER            | 185k <i>claims</i>          | Outcome           |
| Mundo virtual  | ALFWorld         | 3.1k ejemplos               | Outcome           |
| Mundo virtual  | Minecraft        | 58 <i>items</i>             | Outcome           |
| Navegación web | WebShop          | 1.8k ej., 12k instrucciones | Outcome           |
| Navegación web | WebArena         | 812 tareas                  | Outcome           |
| Código         | HumanEval        | 164 problemas               | Outcome           |

| Dominio      | Benchmark | Tamaño                  | Evaluación     |
|--------------|-----------|-------------------------|----------------|
| Código       | SWE-Bench | 2k problemas            | Outcome        |
| Código       | DevAI     | 55 tareas               | <b>Process</b> |
| Multi-agente | PARTNR    | 100k tareas, 5k objetos | Outcome        |

Nótese lo que el propio material resalta: de 10 *benchmarks* citados, **solo uno (DevAI) evalúa el proceso** — el resto solo puntúa si la respuesta final fue correcta o no.

## 5.3 Los números — comparación de métodos en 3 benchmarks

| Método       | HotpotQA | ALFWorld   | WebShop | Rondas                |
|--------------|----------|------------|---------|-----------------------|
| Act          | 29%      | 28%        | 34%     | 1                     |
| CoT          | 29%      | —          | —       | Multi                 |
| ReAct        | 28%      | 40%        | 35%     | Multi                 |
| Reflexion-R1 | 33%      | 48%        | 43%     | 1 + <i>replay</i>     |
| Reflexion-R2 | 40%      | 52%        | 46%     | 2 + <i>replay</i>     |
| Reflexion-R3 | 40%      | 54%        | 48%     | 3 + <i>replay</i>     |
| ExpeL        | 39%      | 59%        | 41%     | Multi + <i>replay</i> |
| AdaPlanner   | —        | 63%        | —       | Multi                 |
| AutoGuide    | —        | <b>79%</b> | 46%     | Multi + <i>replay</i> |

(R1/R2/R3 = 1, 2 o 3 reintentos de Reflexion; "—" = no evaluado en ese benchmark; "+ replay" = además reutiliza experiencia guardada de tareas anteriores.)

**La lectura que propone el material, mirando las filas R1→R2→R3 de Reflexion:** *"casi toda la ganancia está en la primera reflexión"* — de R1 a R2 hay un salto grande (33%→40% en HotpotQA), pero de R2 a R3 el HotpotQA se estanca en 40% — **rendimientos marcadamente decrecientes** después del primer reintento reflexivo.

## 5.4 El precio de reflexionar

| Método    | Costo computacional | Transferible | Tiempo real |
|-----------|---------------------|--------------|-------------|
| Act / CoT | Bajo                | Sí           | ✓           |
| ReAct     | Medio               | Sí           | ✓           |

| Método     | Costo computacional | Transferible                                                      | Tiempo real |
|------------|---------------------|-------------------------------------------------------------------|-------------|
| Reflexion  | Alto                | Requiere <i>pool</i> (compartir el mismo almacén de experiencias) | X           |
| ExpeL      | Alto                | Requiere <i>pool</i>                                              | X           |
| AdaPlanner | Medio               | Sí                                                                | ✓           |
| AutoGuide  | Alto                | Requiere <i>pool</i>                                              | X           |

**El patrón que señala el material:** *"todo lo que aprende de verdad sale del tiempo real y necesita un pool de experiencia."* Hay una excepción explícita: **AdaPlanner** — usa *prompts* en estilo código, sin sacrificar latencia, logrando 63% en ALFWorld (el segundo mejor resultado de la tabla) sin pagar el costo de "tiempo real X" que pagan Reflexion/ExpeL/AutoGuide.

### 5.5 ¿Y si el agente no sabe que está equivocado? — el hallazgo incómodo

|                                                                   |                                                                  |
|-------------------------------------------------------------------|------------------------------------------------------------------|
| Feedback de un LLM promptado<br>(auto-evaluación sin verificador) | Feedback externo confiable<br>(tests, compilador, base de datos) |
| X                                                                 | ✓                                                                |
| Ningún trabajo previo lo demuestra<br>de forma limpia             | Funciona de forma consistente                                    |

Esta es la cita central de **Kamoi et al., TACL 2024** — verificada externamente para este documento: el paper original concluye textualmente que *"no prior work demonstrates successful self-correction with feedback from prompted LLMs, except for studies in tasks that are exceptionally suited for self-correction"*. El material añade el diagnóstico metodológico de por qué tantos estudios previos *parecen* mostrar éxito: **usan oráculos para decidir cuándo parar** — es decir, el experimento sabe de antemano cuál es la respuesta correcta y detiene el *loop* justo cuando la alcanza, lo cual sobreestima artificialmente cuánto "sabe" el agente que se ha corregido a sí mismo.

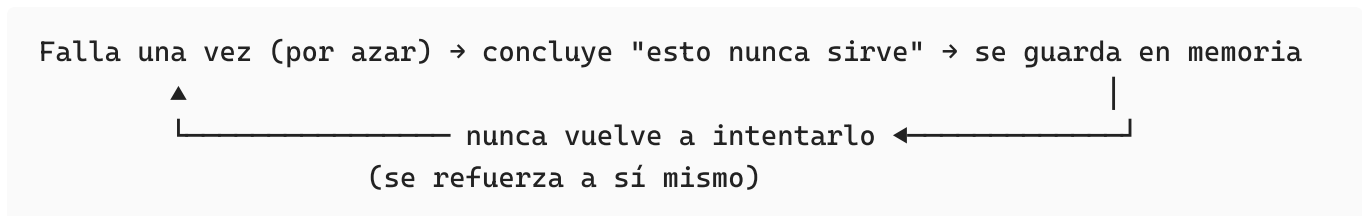
**La tabla que hace el punto con números concretos (SCoRe, ICLR 2025, sobre el benchmark MATH):**

| Método (MATH)           | Acc.@t1 | Acc.@t2 | $\Delta(t1, t2)$ |
|-------------------------|---------|---------|------------------|
| Gemini 1.5 Flash (base) | 52.6%   | 41.4%   | -11,2%           |
| Self-Refine             | 52.8%   | 51.8%   | -1,0%            |
| SCoRe (entrenado)       | 60.0%   | 64.4%   | +4,4%            |

( $\text{Acc.}@t1$  = acierto en el primer intento;  $\text{Acc.}@t2$  = acierto tras revisarse;  
 $\Delta(t1, t2) = \text{Acc.}@t2 - \text{Acc.}@t1$  — si es negativo, revisarse dañó el resultado.)

**Lo que dice esta tabla, en palabras del material:** "pedirle a un modelo que se revise, sin más, le hace perder 11 puntos: cambia respuestas correctas por incorrectas." Self-Refine apenas frena el daño (de  $-11,2$  a  $-1,0$ ), pero sigue siendo negativo. **Solo el modelo entrenado específicamente para auto-corregirse (SCoRe) obtiene una ganancia positiva**, y es de apenas  $+4,4$  puntos — un resultado modesto que confirma, con números, la tesis crítica de Kamoi et al.: la auto-corrección sin verificador externo no es gratis, y a menudo **es contraproducente**.

## 5.6 El riesgo de recordar una conclusión falsa



"Si la reflexión diagnóstica mal y esa conclusión se guarda, el agente deja de explorar una vía que sí funcionaba. La memoria convierte un error puntual en uno permanente." Esta es la cita del paper "**Honest Lying: Understanding Memory Confabulation in Reflexive Agents**" (arXiv:2605.29463, 2026).

**Investigación complementaria — verificación y detalle adicional de este paper (Dixit, Kamal & Oates, ICML 2026 Workshop "Failure Modes in Agentic AI"):** el paper examina agentes tipo Reflexion en **ALFWorld** y **HumanEval**, y muestra que estos agentes pueden almacenar interpretaciones **seguras pero incorrectas** de la tarea, y seguir actuando sobre ellas en intentos sucesivos — incluso cuando el entorno se reinicia limpio en cada intento. Los autores definen este modo de fallo como "**confabulación de memoria**" y proponen una métrica, la *Reflection Repetition Rate* (RRR, tasa de repetición de reflexión), que detecta cuándo un agente sigue apoyándose en contenido reflexivo incorrecto. Usando RRR identificaron **16 entornos "congelados"** en ALFWorld donde ninguna de 121 reflexiones generadas mencionaba el objeto objetivo correcto. Su mitigación — reemplazar el auto-diagnóstico abierto por una extracción **programática** de señales de fallo a nivel de trayectoria — subió la mención correcta del objeto de 0% a 86% y bajó la RRR de 0.64 a 0.10, resolviendo 3 de los 16 entornos congelados. Es evidencia concreta y reciente (mayo de 2026) de que el riesgo señalado en la diapositiva no es hipotético — ya está documentado y cuantificado.

## 5.7 Checklist antes de confiar en tu loop (diapositiva 42, del propio material)

1. ¿De dónde viene la señal? ¿Es verificable?

2. ¿Quién decide cuándo parar? ¿Se usa la respuesta correcta para decidirlo? (*el sesgo de "oráculo" de Kamoi et al.*)
3. ¿El *baseline* recibió el mismo presupuesto de cómputo?
4. ¿Se midió el  $\Delta$  **con signo**? Una segunda pasada puede restar, no solo sumar.
5. Si se guarda la lección: ¿se puede revocar cuando resulte falsa? (*el riesgo de confabulación de memoria, §5.6*)

**Regla práctica del material:** *"Si no tienes verificador externo, invierte en mejores prompts antes que en más iteraciones."*

## 6. De corregir a mejorarse — tres escalas del mismo mecanismo

|                     |   |                       |   |                      |
|---------------------|---|-----------------------|---|----------------------|
| Clase actual (10)   |   | Clase siguiente (11)  |   | Clase siguiente (12) |
| Corregir una salida | → | Reescribir el prompt  | → | Evolucionar una      |
| población           |   |                       |   |                      |
| (efímero)           |   | (persistente,         |   | (colectivo, muchos   |
|                     |   | la lección se guarda) |   | candidatos en        |
| paralelo)           |   |                       |   |                      |

*"Es el mismo mecanismo en tres escalas. Self-Refine y Reflexion son solo el primer escalón."*  
 Esta diapositiva funciona como mapa de todo el Módulo 7: la Sesión 10 (esta) cubre corregir una salida puntual; la Sesión 11 sube de escala a modificar el *prompt* del sistema de forma persistente a partir de trazas acumuladas; la Sesión 12 sube otra escala más, a optimizar una **población** de agentes en paralelo (el mismo problema de coordinación/alineación/estabilidad multiagente anticipado en §3.3).

### 6.1 ¿Qué puede cambiar un agente de sí mismo? — el eje $\Sigma$ vs. $\theta$

|                        | Modelo base ( $\theta$ )                                                                                                       | Andamiaje ( $\Sigma$ )                                   |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------|
| <b>Mecanismo</b>       | <b>SFT, RL, DPO</b> ( <i>Direct Preference Optimization</i> , optimización directa de preferencias) sobre datos auto-generados | <i>Prompts</i> , memoria, herramientas, control de flujo |
| <b>Costo/velocidad</b> | Lento, costoso, cambio global                                                                                                  | Rápido, reversible, cambio local                         |

**Definiciones del material** (relevantes porque son términos que reaparecerán en las Sesiones 11-12):

- **SFT**: reentrenar el modelo con ejemplos de entrada y salida correcta.
- **RL**: ajustar los pesos del modelo maximizando una recompensa numérica.
- **DPO**: ajustar con pares "esta respuesta es mejor que aquella", sin entrenar un modelo de recompensa aparte.

"Casi todo lo accesible hoy está en  $\Sigma$ : no hace falta reentrenar para que el agente mejore." (Ren et al., "Self-Improvements in Modern Agentic Systems", arXiv:2607.13104, 2026, §3.2). El material cierra esta sección con una consecuencia directa para el resto del programa: **las Sesiones 11 y 12 viven enteras dentro de la caja  $\Sigma$**  — es decir, todo lo que sigue en el Módulo 7 asume que el estudiante **no** va a reentrenar un modelo, sino a trabajar sobre *prompts*, memoria y orquestación.

**Regla del pulgar del material:** "Empieza siempre por  $\Sigma$ . Pasa a  $\theta$  solo cuando el andamiaje ya no dé más de sí." — es la misma lógica de "empezar por lo barato y reversible" que ya apareció en el checklist Workflows-vs-Agents de Anthropic (Sesión 15) y en los "3 caminos" de multimodalidad (Sesión 17): en todos los casos, el programa recomienda **agotar la opción de orquestación antes de pagar el costo de entrenar/ajustar un modelo**.

## 7. Práctica — construyendo el loop (dos agentes en LangGraph)

El material describe dos agentes que se implementarían en un notebook (notebooks/10\_feedback.ipynb, referenciado en las diapositivas 50-51 como "Parte 1" y "Parte 2") **que no está presente en esta carpeta del repositorio** — solo se cuenta con el PDF de la sesión; a diferencia de las Sesiones 17 y 18, aquí no hay código de laboratorio adjunto para trazar contra la teoría.

### 7.1 Agente 1 — Self-Refine (feedback interno)

```

generar → criticar → refinar → ¿ok? —no→ (vuelve a generar)
                        |
                        sí → fin

```

"Es la fila roja del recap: barato y poco confiable. El propio LLM decide cuándo parar."

### 7.2 Agente 2 — Reflexion + intérprete (feedback externo)

```

escribir código → ejecutar tests → ¿pasa? —sí→ fin
                        |
                        no

```

"Dos cambios respecto al agente 1: el juez es el intérprete [no el LLM], y la lección persiste entre problemas." Es, en términos de la taxonomía de §3, la diferencia exacta entre **Interno intra-tarea** (Agente 1) y una combinación de **Externo** (el verificador: el intérprete que ejecuta tests) con **Interno inter-tarea** (la reflexión que se guarda en memoria entre problemas).

## 7.3 Experimento guiado propuesto

1. Agente 1 con y sin la etapa de crítica → ¿mejora real o solo consume más *tokens*?
2. Agente 2 con memoria vaciada entre problemas → ¿sirve realmente la memoria inter-tarea?
3. Quitar el intérprete y dejar que el LLM juzgue → ¿cuántas veces se equivoca?

"El paso 3 reproduce el hallazgo de Kamoi et al.: cuando el juez falla, el loop optimiza hacia el lugar equivocado." — el ejercicio está diseñado para que el estudiante reproduzca **empíricamente, con su propio código**, la misma conclusión que motivó la sección de "letra pequeña" (§5), en vez de solo leerla en una diapositiva.

## 8. Cierre de la sesión

### 8.1 Lo que queda abierto (diapositiva 54)

El material lista 6 direcciones de investigación abiertas según Liu et al. (IJCAI-25, §6 — *Challenges and Future Directions*): *feedback* multimodal, tiempo real multi-agente, meta-aprendizaje adaptativo, explicabilidad, *sim-to-real*, y estandarización — esta última con una mención concreta a protocolos emergentes para agentes heterogéneos: **MCP** (*Model Context Protocol*), **A2A** (*Agent-to-Agent*), **ANP** (*Agent Network Protocol*) y **Agora**. El material señala que el primero de la lista, *tiempo real multi-agente*, es literalmente el tema de la Sesión 12.

### 8.2 Para llevarse — las 4 frases de síntesis del propio material

1. El *feedback* convierte agentes reactivos en adaptativos.
2. Hay cuatro fuentes de señal: lo barato no es confiable, lo confiable no es barato.
3. Sin verificador externo, desconfía de la auto-corrección.
4. Corregir una salida es el primer escalón: sigue reescribir el *prompt*.

**Puente a la siguiente clase:** "Tenemos miles de trazas de agentes fallando. ¿Cómo las convertimos en un *prompt* mejor?" — la pregunta que abre, explícitamente, la Sesión 11.

---

## 9. Síntesis propia — cómo encaja esta sesión en el resto del programa

1. **Esta sesión formaliza, con literatura académica revisada por pares, una intuición que ya venía apareciendo de forma implícita desde el Módulo 5:** el "criterio de costo del error" del *checklist* de Anthropic (Sesión 15) y la elección entre modelos de visión local vs. cloud (Sesión 17, comparación LLaVA vs. GPT-4o-mini) son, en retrospectiva, casos particulares de la misma pregunta que estructura esta clase — **¿de dónde viene la señal que le dice al sistema si acertó o no, y qué tan confiable es esa señal?**
2. **El eje  $\Sigma$  (andamiaje) vs.  $\theta$  (modelo)** es, en efecto, el mismo eje "barato/reversible vs. costoso/global" que ya organizó los "3 caminos" de multimodalidad (Sesión 17: LLM+Tools vs. LLM+Adapters vs. Unified Models) y el framework Workflows-vs-Agents (Sesión 15) — el programa vuelve a esta misma disyuntiva en cada módulo, aplicada a un problema distinto cada vez.
3. **El hallazgo de Kamoi et al. + la tabla de SCoRe (§5.5) son, hasta ahora, la evidencia más rigurosa y cuantificada de todo el curso** de que una técnica intuitivamente atractiva ("dejar que el agente se revise a sí mismo") puede **empeorar** el resultado en vez de mejorarlo — un contrapeso necesario frente al entusiasmo generalizado del Módulo 5-6 por dar cada vez más autonomía a los agentes.
4. **La práctica de esta sesión (§7) es, en diseño, un experimento controlado** — no solo un ejercicio de código: está construida específicamente para que el estudiante reproduzca con datos propios la misma conclusión que el material argumenta con literatura ajena, cerrando el círculo entre teoría y evidencia.

---

## 10. Checklist práctico — diseñando un mecanismo de feedback para un agente

- ☐ ¿De cuál de las 4 fuentes (interno, externo, multi-agente, humano) viene la señal de éxito/fallo? ¿Es la más barata que aún resulta confiable para esta tarea?
- ☐ Si la señal es "interno" (el propio LLM se juzga): ¿existe evidencia de que la auto-corrección funciona en *esta* tarea específica, o se está asumiendo sin verificar? (Regla de Kamoi et al.: por defecto, desconfiar.)
- ☐ ¿Existe una señal externa verificable "escondida" que no se está aprovechando — un test, un compilador, una base de datos, un cálculo que ya existe en el sistema?
- ☐ ¿Quién decide cuándo el *loop* de corrección para? ¿Ese criterio usa, aunque sea indirectamente, la respuesta correcta (sesgo de oráculo)?



- ☐ ¿Se midió el efecto de la corrección con un  $\Delta$  **con signo**, comparando contra un *baseline* con el mismo presupuesto de cómputo?
  - ☐ Si el agente guarda lecciones en memoria: ¿hay manera de **revocar** una lección que resulte falsa? ¿Se está midiendo algo como una tasa de repetición de reflexión (RRR) para detectar confabulación de memoria?
  - ☐ ¿La tarea necesita corregir una salida puntual ( $\Sigma$ , escala 1), reescribir el *prompt* del sistema de forma persistente ( $\Sigma$ , escala 2), o evolucionar una población de variantes ( $\Sigma$ , escala 3)? — o de verdad, tras agotar  $\Sigma$ , hace falta tocar el modelo ( $\theta$ )?
  - ☐ Si se está considerando multiagente para obtener "varias perspectivas": ¿hay un plan explícito para los tres problemas conocidos (coordinación, alineación, estabilidad)?
- 

## 11. Referencias

**Del material original (citás completas, Referencias I y II del PDF):**

- Liu, Z., Bai, X., Chen, K., et al. (2025). "A Survey on the Feedback Mechanism of LLM-based AI Agents." IJCAI-25 Survey Track, pp. 10582–10592. — **el paper ancla de toda la sesión.**
- Ren, Z., Chen, Y., Guo, D., et al. (2026). "Self-Improvements in Modern Agentic Systems: A Survey." arXiv:2607.13104.
- Kamoi, R., Zhang, Y., Zhang, N., Han, J., Zhang, R. (2024). "When Can LLMs Actually Correct Their Own Mistakes? A Critical Survey of Self-Correction of LLMs." TACL, 12:1417–1440.
- Yao, S., Zhao, J., Yu, D., et al. (2023). "ReAct: Synergizing Reasoning and Acting in Language Models." ICLR 2023.
- Shinn, N., Cassano, F., Berman, E., et al. (2023). "Reflexion: Language Agents with Verbal Reinforcement Learning." NeurIPS 2023.
- Madaan, A., Tandon, N., Gupta, P., et al. (2023). "Self-Refine: Iterative Refinement with Self-Feedback." NeurIPS 2023.
- Zhao, A., Huang, D., Xu, Q., et al. (2024). "ExpeL: LLM Agents are Experiential Learners." AAAI 2024.
- Wei, J., Wang, X., Schuurmans, D., et al. (2022). "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models." NeurIPS 2022.
- Sun, H., Zhuang, Y., Kong, L., Dai, B., Zhang, C. (2023). "AdaPlanner: Adaptive Planning from Feedback with Language Models." NeurIPS 2023.
- Fu, Y., Kim, D.-K., Kim, J., et al. (2024). "AutoGuide: Automated Generation and Selection of Context-Aware Guidelines for Large Language Model Agents." NeurIPS 2024.

- Zheng, L., Chiang, W.-L., Sheng, Y., et al. (2023). "*Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*." NeurIPS 2023.
- Zhuge, M., Zhao, C., Ashley, D., et al. (2024). "*Agent-as-a-Judge: Evaluate Agents with Agents (DevAI)*." arXiv:2410.10934.
- Kumar, A., Zhuang, V., Agarwal, R., et al. (2025). "*Training Language Models to Self-Correct via Reinforcement Learning (SCoRe)*." ICLR 2025 (Oral), Google DeepMind. arXiv:2409.12917.
- Allard, M.-A., Teinturier, A., Xing, V., Viaud, G. (2026). "*Experiential Reflective Learning for Self-Improving LLM Agents*." arXiv:2603.24639.
- Shi, Y., Li, S., Wu, C., et al. (2025). "*Search and Refine During Think: Facilitating Knowledge Refinement for Improved Retrieval-Augmented Reasoning (AutoRefine)*." NeurIPS 2025. arXiv:2505.11277.
- Dixit, P., Kamal, S., Oates, T. (2026). "*Honest Lying: Understanding Memory Confabulation in Reflexive Agents*." arXiv:2605.29463.
- Wang, G., Xie, Y., Jiang, Y., et al. (2024). "*Voyager: An Open-Ended Embodied Agent with Large Language Models*." TMLR.
- Du, Y., Li, S., Torralba, A., Tenenbaum, J., Mordatch, I. (2024). "*Improving Factuality and Reasoning in Language Models through Multiagent Debate*." ICML 2024.
- Ouyang, L., Wu, J., Jiang, X., et al. (2022). "*Training Language Models to Follow Instructions with Human Feedback*." NeurIPS 2022.
- Repositorio del survey: [github.com/kevinson7515/Agents-Feedback-Mechanisms](https://github.com/kevinson7515/Agents-Feedback-Mechanisms)

### Investigación complementaria (verificación añadida en este documento):

- Se confirmó externamente la existencia y el contenido exacto de las tres citas más importantes del material: el survey de Liu et al. (IJCAI-25, DOI [10.24963/ijcai.2025/1175](https://doi.org/10.24963/ijcai.2025/1175)), el *critical survey* de Kamoi et al. (TACL 2024, [aclanthology.org/2024.tacl-1.78](https://aclanthology.org/2024.tacl-1.78)) y el paper de confabulación de memoria (Dixit, Kamal & Oates, arXiv:2605.29463, aceptado en el *ICML 2026 Workshop on Failure Modes in Agentic AI*) — las tres existen, están correctamente atribuidas y respaldan exactamente las afirmaciones que el material les asigna.
- Detalle adicional del paper de confabulación de memoria (§5.6): la métrica *Reflection Repetition Rate* (RRR), los resultados en 16 entornos "congelados" de ALFWorld (0%→86% de mención correcta del objeto tras la mitigación propuesta, RRR de 0.64→0.10), no descritos en el material original de la clase pero directamente relevantes para el *checklist* de la sesión (§5.7, punto 5).
- Arco interno del curso: *Sesion15\_LangGraph\_MultiAgent\_ANALISIS\_COMPLETO.md* (Módulo 5) — el eje  $\Sigma/\theta$  de esta sesión es la misma lógica "barato/reversible vs. costoso/global" que ya organizó el *checklist* Workflows-vs-Agents de Anthropic; *Sesion17\_Multimodalidad\_ANALISIS\_COMPLETO.md* (Módulo 6) — los "3 caminos" de

multimodalidad (LLM+Tools/Adapters/Unified Models) son la misma disyuntiva de costo de ingeniería aplicada a otro problema.

---

*Documento generado a partir del PDF de la Sesión 10 (Módulo 7, UTEC Posgrado) — 58 diapositivas con texto extraído directamente + ~20 diapositivas clave renderizadas para verificar diagramas — más verificación externa de las tres citas académicas centrales del material.*